

InfoRank: Unbiased Learning-to-Rank via Conditional Mutual Information Minimization

Jiarui Jin^{1,*}, Zexue He^{2,*}, Mengyue Yang³, Weinan Zhang¹, Yong Yu¹, Jun Wang³, Julian McAuley²

¹Shanghai Jiao Tong University, ²University of California San Diego, ³University College London

{jinjiarui97,wnzhang,yyu}@sjtu.edu.cn,{mengyue.yang,jun.wang}@ucl.ac.uk,{zehe,jmcauley}@eng.ucsd.edu

ABSTRACT

Ranking items regarding individual user interests is a core technique of multiple downstream tasks such as recommender systems. Learning such a personalized ranker typically relies on the implicit feedback from users' past click-through behaviors. However, collected feedback is biased toward previously highly-ranked items and directly learning from it would result in "rich-get-richer" phenomena. In this paper, we propose a simple yet sufficient unbiased learning-to-rank paradigm named InfoRank that can simultaneously mitigate the effects of both *position* and *popularity* biases. To this end, we first summarize the impacts of those biases into a single *observation* factor, therefore providing a unified treatment of the bias problem. We then minimize the mutual information between the *observation* estimation and the *relevance* estimation conditioned on the input features. By doing so, our relevance estimation can be proved to be free of bias. To realize InfoRank, we first adopt an attention mechanism to capture hidden correlations among user-item features and produce the estimations of *observation* and *relevance*. We then introduce a regularization term based on conditional mutual information to encourage our relevance estimation to be conditionally independent of the *observation* estimation. Experiments over three large-scale recommendation and search datasets demonstrate that InfoRank learns more accurate and unbiased ranking strategies from biased feedback compared to strong baselines under a variety of user browsing patterns.

KEYWORDS

Unbiased Learning-to-Rank, Conditional Mutual Information, Position Bias, Popularity Bias

1 INTRODUCTION

Implicit feedback from user interactions (such as clicks) is a common data source, in settings where "ground-truth" about explicit relevance is difficult to obtain. Although implicit feedback alleviates data labelling costs, it introduces a range of data bias issues [4, 23]. As summarized in [10], typical biases in the collection phase and the serving phase are *position bias* and *popularity bias* respectively, which are intensified by the feedback loops of ranking systems, as

illustrated in Figure 1(a). Concretely, *position bias* occurs when collecting user feedback, which is caused by users' browsing patterns. As Figure 1(b) shows, a user typically *observes* the presented item list from top to bottom, during which the user's attention declines rapidly [22, 23]; thus higher-ranked items receive more exposure and more opportunity to be *observed* and further clicked [13, 14] and more opportunity to be collected as implicit feedback. In contrast, *popularity bias* happens when the system returns ranked lists to serve users, as shown in Figure 1(b), which leads a ranking system to recommend popular items more frequently than their popularity would warrant. In both cases, blindly optimizing ranking performance on implicit feedback data may reinforce the existing presentation or popularity order, rather than learning personalized relevance, since the higher-ranked or popular items would receive more *observations* from users and get more opportunities to gain the positive feedback (e.g., clicks) from users.

Recently, several works have raised awareness of such bias issues and sought to discover the relevance hidden in biased feedback. For position bias, recent studies leverage counterfactual learning [28], where the position is treated as a counterfactual factor [33] and user feedback is debiased through inverse propensity weighting (IPW) [2, 21, 43]. For popularity bias, recent attempts develop a diversity-based regularization approach [1, 26] to balance rating and viewpoint values, or adversarial learning [27] to learn the implicit association between popular and unpopular items. However, these approaches typically account for either position or popularity factors in isolation, and lack a unified debiasing method.

In this paper, we reveal that these common biases*, occurring during either collection or serving phase, caused by either users or systems, affect ranking performance through the *observation* factor. For example, position bias arises from the user side, since the position is associated with *whether the item has been observed or not*, and thus influences user feedback; popularity bias caused by the ranker, since the generated list of a ranking system is influenced by *how many times the item has been observed (or further clicked) by users*. This insight motivates us to mine user-item relevance free from the effect of the *observation* factor, since user-item features should be the determining factors for relevance. That is, regardless of the item's position and how many times it has been observed before, the relevance of the item to the user ought to be the same.

To this end, we propose InfoRank, a novel unbiased learning-to-rank paradigm that allows us to model the relevance from rich user-item profiles while mitigating the biases raised by the *observation* factor. Specifically, we first introduce an attention mechanism to discover and capture hidden correlations among the rich user-item features. We provide an illustrated example in Figure 1(c):

*Equal contributions: Jiarui and Zexue.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Pre-print, Under review, 2023

© 2023 Association for Computing Machinery.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

*For simplicity, we describe our work in the context of position and popularity biases which are classic biases during collection and serving phases; one can extend it to other biases, which we leave as future work.

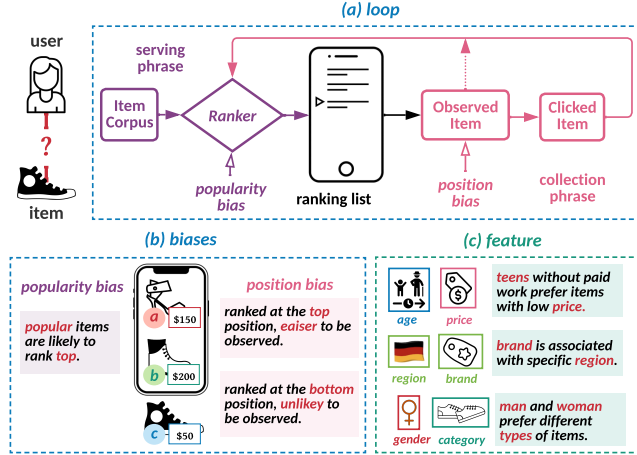


Figure 1: An illustrated example of the feedback loop and biases in learning-to-rank. For each item, a ranking system combines user and item features (c) with implicit feedback to generate the ranking list. However, this system suffers from popularity bias and position bias (b). Moreover, biases will be amplified in loop (a), which may result in a “rich-get-richer” dilemma. In this paper, we formulate the impact of those biases from an *observation* factor.

When a teen searches for shoes, the ranking system returns several shoes based on her profile and historic data. Supposing she has a limited budget (*part of a user feature*), she is more likely to prefer shoes with a low price (*an item feature*). These correlations are essential to discover users’ preferences toward the given items. In order to ensure the estimated relevance is solely governed by user-item features, we then introduce the *observation* factor as a latent variable, and encourage the unbiased estimation (i.e., relevance) to be independent of the biased factor (i.e., observation) conditioned on the input features by minimizing their mutual information. We further derive a regularization formulation of the conditional mutual information minimization, and correspondingly design a novel end-to-end framework to simultaneously optimize the ranking performance and benefit the ranker in being free of different kinds of biases raised from implicit feedback.

Experiments on three datasets demonstrate the superiority of InfoRank over the state-of-the-art under various browsing patterns. Further ablation studies illustrate that our idea of conditional mutual information minimization can also promote the performance of other ranking models.

2 BRIDGING BIASES IN RANKING TO DEPENDENCE IN CAUSALITY

2.1 Biased and Unbiased Learning-to-Rank

The goal of a point-wise learning-to-rank algorithm is to train a ranker f that assigns a relevance score to each query-item (so-called user-item) pair. However, since explicit relevance signals are usually too expensive in practice, biased ranking algorithms directly replace them with implicit click signals. Let u denote the user and d denote the item. Let $\mathbf{x}$ denote the feature vector of u and d . Let $\mathcal{D}$ denote the whole set of items and $\mathcal{D}_u$ denote the set of items associated with u . c represents whether d is clicked or not.

Table 1: A summary of notations.

Notations	Explanations
O, C	Observation, click attributes (implicit feedback)
$X = (U, I)$	Features (user and item side)
R	Relevance attribute (unbiased estimation)
$\mathcal{I}$	Conditional mutual information, see Eq. (14)
$\mathcal{D}, \mathcal{D}_u$	Dataset, dataset associated with user u

The risk function in learning is defined as

$$\mathcal{F}(f) = \sum_u \sum_{d \in \mathcal{D}_u} \Delta(f(\mathbf{x}), c), \quad (1)$$

where $\Delta(f(\mathbf{x}), c)$ denotes a point-wise loss function. Traditionally, the ranker is learned from user browsing logs including labeled click data to find the optimal ranker that minimizes the risk function. However, it suffers from position bias [4, 25] and popularity bias [1, 26]. We call this *biased* learning-to-rank. Recent attempts have proposed *unbiased* learning-to-rank algorithms aiming to eliminate such biases in click data and train an unbiased ranker.

Position Bias. Several prior works [4, 20, 22, 43, 44] reveal that items with higher positions are more likely to be *observed*, and further clicked. Hence, training a ranker directly on click data would lead it to estimate the position order instead of the personalized relevance. To address this bias, prevailing debiasing methods usually introduce *relevance*[†] r to represent whether d is relevant or not, and estimate r instead of c in ranking. They leverage the fact that a user *clicks* an item only when it is both *observed* and perceived as *relevant*, which can be formulated as:

$$P(C = 1 | X = \mathbf{x}) = P(R = 1 | X = \mathbf{x}) \cdot P(O = 1 | X = \mathbf{x}). \quad (2)$$

where $O^\ddagger$ denotes the random variable. Then, the aim of unbiased learning-to-rank is to estimate relevance from click data and provide a ranked list according to $P(R = 1 | X = \mathbf{x})$ different from biased learning-to-ranking using $P(C = 1 | X = \mathbf{x})$.

We note that in previous approaches [13, 47], the relevance is estimated from user-item features while the observation is estimated from bias-related features (e.g., position). Since our paper’s primary objective is not to separate $\mathbf{x}$ into these two components, for the sake of simplicity, we utilize $\mathbf{x}$ in Eq. (2) and our following analysis.

Popularity Bias. Several previous works [1, 15, 26, 50] show that items with higher popularity are more likely to be posted, and further be *observed* and clicked. Hence, directly optimizing the performance of a ranker on click data would lead it to estimate the popularity order instead of the personalized relevance. As position bias is one factor of popularity bias [50], many debiasing approaches [35, 45, 50] also estimate $P(R = 1 | X = \mathbf{x})$ instead of $P(C = 1 | X = \mathbf{x})$.

[†]For convenience, we only consider binary relevance here. One can easily extend our framework to a multi-level relevance case, with Eq. (25) as a possible solution to convert it to the binary setting [4, 20].

[‡]We use uppercase letters (i.e., C, R, O, X) to denote random variables (see Table 1 for explanations), and lowercase letters (i.e., $c, r, o, \mathbf{x}$) to denote the corresponding value for each data point d . We further expand the random variables to represent a set of data points and use calligraphic letters (i.e., $\mathcal{C}, \mathcal{R}, \mathcal{O}$) as notation. In other words, to study the case of user u and item d , $C = c, R = r, O = o$ where $c, r, o \in \{0, 1\}$ show whether d is clicked, relevant, observed by u , while $C = \{c = 1\}_d, \mathcal{R} = \{r = 1\}_d, \mathcal{O} = \{o = 1\}_d$ show previous users’ click/relevance/observation feedback on d .

Considering that these algorithms use the previous user feedback (especially clicks) of an item d for popularity estimation, we use (C, O, R) to denote the set of previous feedback (click, observation, relevance) on d . Take C vs. C as an example (O and R have similar interpretations): While C is a factor measuring how user u will consume the item d , C is a factor denoting how the item d is previously consumed by other users. Therefore, different from C whose value is the corresponding click $c = 1$ that u gives to d , the instance of C is a set of previous click feedback associated with d , denoted as $\{c = 1\}_d$.

When a ranker estimates relevance between u and d , it is free of popularity bias if the current relevance estimation is independent of the item's previous click history conditioned on its features, which can be formulated as

$$P(R = 1|C = \{c\}_d, X = \mathbf{x}) = P(R = 1|X = \mathbf{x}). \quad (3)$$

Note that $\{c = 1\}_d$ doesn't include the "current click" that u gives to d (i.e., the one to be estimated).

2.2 Causality in Ranking

According to the analysis above, position and popularity biases exist because (i) user observation feedback is influenced by the position; and (ii) a system generates the ranked list according to observations or further clicks which may be biased. Therefore, the *observation factor* introduces existing biases into downstream factors. To this end, we regard the *observation factor* as the "sensitive attribute".

In other words, an ideal ranker should satisfy: for any user u and item d , given their associated feature vector $\mathbf{x}$,

$$P(R = r|O = 1, X = \mathbf{x}) = P(R = r|O = 0, X = \mathbf{x}) \quad (4)$$

holds for all the relevance score $r \in \{0, 1\}$, and any observation value $o \in \{0, 1\}$ attainable by O . We note that Eq. (4) can be regarded as the *conditional independence* between the hidden factors O and R , where the external factors (e.g., user, query) are aggregated into X . From Eq. (4), one can easily derive its corresponding evaluation metric as follows:

$$\Delta CI := |P(R = 1|O = 1, X = \mathbf{x}) - P(R = 1|O = 0, X = \mathbf{x})|. \quad (5)$$

It is easy to show that if Eq. (4) holds, $\Delta CI = 0$.

We then respectively connect this conditional independence to the above biases as follows.

Causality in Position Bias. The basic idea to address the position bias is to estimate the bias-free relevance R , where R is expected to be solely governed by user-item features X . However, we argue that applying Eq. (2) is not enough to achieve this, as the estimation of an item's relevance can still be influenced by whether it has been observed or not. Therefore, we propose to further assure the conditional independence between R and O .

By combining Eq. (2) and Eq. (4), we have

$$P(R = 1|X = \mathbf{x}) = P(R = 1|O = o, X = \mathbf{x}). \quad (6)$$

Causality in Popularity Bias. According to Eq. (2), we can derive $P(C = \{c = 1\}_d|X = \mathbf{x}) = P(R = \{r = 1\}_d|X = \mathbf{x}) \cdot P(O = \{o = 1\}_d|X = \mathbf{x})$, which means given the feature of an item d , then its previous clicks (i.e., $\{c = 1\}_d$) only occur when d is both relevant (i.e., $\{r = 1\}_d$) and observed (i.e., $\{o = 1\}_d$) by users.

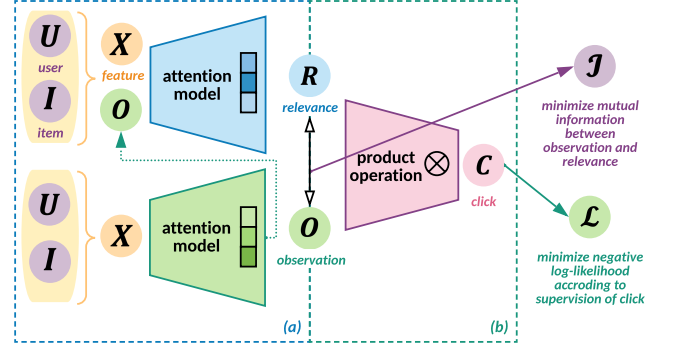


Figure 2: The overall architecture of InfoRank, where (a) we first mine correlations between user-item features through an attention mechanism (Section 3.2); and (b) we then formulate conditional mutual information in a regularization form (i.e., $\mathcal{J}$) to make relevance conditionally independent of the observation factor (Section 3.3), which we incorporate with supervisions over user behaviors (i.e., $\mathcal{L}$) to capture the relevance behind biased feedback (Section 3.4). We note that InfoRank also works when there is no observation information in user browsing logs, where we substitute the real observations by the estimated ones.

We can then further derive:[§]

$$\begin{aligned} P(R = 1|C) &\stackrel{(1)}{=} \frac{P(C|R = 1)P(R = 1)}{P(C)} \\ &\stackrel{(2)}{=} \frac{P(O|R = 1)P(R = 1)}{P(O)P(R = 1)} P(R = 1|R) \stackrel{(3)}{=} \frac{P(R = 1|O)}{P(R = 1)} P(R = 1|R), \end{aligned} \quad (7)$$

where sub-equations (1)(2)(3) hold due to Bayes' theorem.

Noticing that $O = \{o = 1\}_d$ and $O = 1$ have strong correlation as they both represent user observations, we argue that reinforcing $O = 1$ and $R = 1$ independent conditioned on $X = \mathbf{x}$ can approximately make $P(R = 1|O = \{o = 1\}_d)/P(R = 1)$ close to 1. As $\mathcal{R} = \{r = 1\}_d$ and $R = 1$ share different representation space but represent the same factor (i.e., relevance), $P(R = 1|\mathcal{R} = \{r = 1\}_d)$ indicates the inductive ability of the ranker, which corresponds to the process of learning from $\mathcal{R} = \{r = 1\}_d$ to infer $R = 1$: using item d 's previous users' relevance feedback to infer the current behavior of u on d .

Remark. Combining Eqs. (6) and (7), pushing $O = 1$ and $R = 1$ independent conditioned on $X = \mathbf{x}$ would contribute to both addressing position bias and popularity bias.

2.3 Connections to Related Work

Implicit feedback such as clicks is abundant and easy to collect [22, 30, 48]. However, implicit feedback is known to suffer from various biases [19, 23]. Several previous works focus on developing unbiased ranking methodologies for position bias, which can be roughly categorized into two streams. One stream is based on some basic assumptions about user browsing behaviors [3, 9, 14, 16, 39, 40]. These models maximize the likelihood of the observed user behavior in historical data collected from browsing logs. For example,

[§]For simplicity, in Eq. (7), we use C , O , and $\mathcal{R}$ to denote $C = \{c = 1\}_d$, $O = \{o = 1\}_d$ and $\mathcal{R} = \{r = 1\}_d$, and we also omit the condition $X = \mathbf{x}$.

Jin et al. [22] adopted survival analysis techniques to model user browsing behavior. The other stream follows counterfactual learning [4, 25, 44] which treats click bias as a counterfactual factor [33] and debiases user feedback through inverse propensity weighting [20]. For example, Wang et al. [42] derived a new weighting scheme taking a holistic treatment of both clicks and non-clicks. In addition, Tran et al. [36] presented a collection of these algorithms, and Oosterhuis and de Rijke [28] investigated to the use of online interventions for unbiased ranking. Popularity bias has also been the subject of recent research. Kamishima et al. [26] proposed an information-neutral recommender by guaranteeing the independence between the ranker outcome and viewport features. For example, Krishnan et al. [27] incorporated an adversarial network to play a min-max game between a recommender and an adversary to learn an implicit association between popular and unpopular items. We also note that there is recently proposed work [46] jointly addressing multiple biases.

However, each of these methods focuses on specific designs based on different user simulations (e.g., contextual PBM in [46]). Instead, our paper seeks to unify different biases via a single observation factor, which allows us to simultaneously reduce the effect of different biases in a simple and indirect way.

3 THE INFORANK FRAMEWORK

3.1 Overview

As previous section reveals the connections between the conditional independence and the position and popularity biases, the basic idea of InfoRank is to learn unbiased ranking strategies through joint optimization of the ranking performance and the conditional independence. As depicted in Figure 2, we first use an attention mechanism to discover the latent correlations between user-item features and generate probability distributions $P(R = 1|O, X)$ and $P(O = 1|X)$ (shown in (a)). Then, we formulate the conditional mutual dependence into a regularization term (shown as $\mathcal{I}$), and derive the probability distribution $P(C = 1|X)$ through a product operation (shown in (b)) according to Eqs. (2) and (6), where clicks can be supervised by biased feedback (shown as $\mathcal{L}$).

3.2 Unbiased Estimation

3.2.1 Estimating R and O . A standard way of aggregating the user-item features is to directly apply Multi-Layer Perceptron (MLP) layers, as described in previous prevailing user modelling algorithms [11, 12, 34]. However, as stated in [31, 49], these models are not expressive to effectively capture the rich correlations between the user-item features. Taking Figure 1 as an example, a teen might focus on price when selecting shoes, implying a correlation between age and price. This motivates us to jointly consider the information of both the user and item sides to weigh each feature instead of solely accounting for only one side.

Hence, we introduce a multi-head attention mechanism [37, 38]. Formally, let $\mathbf{x} = \{\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{N-1}\}$ denote N categorical user-item features, where $\mathbf{x}_n$ is a one-hot vector representing the n -th feature in $\mathbf{x}$. For non-categorical features, we directly feed them into a neural network to get its $\mathbf{x}_n$ s. Then, for the h -th head we have:

$$\beta_{ij}^{(h)} = (\mathbf{x}_i \mathbf{W}_T^{(h)}) \cdot (\mathbf{x}_j \mathbf{W}_S^{(h)})^\top, \quad (8)$$

where $\mathbf{x}_i, \mathbf{x}_j$ are the i -th and j -th feature, $\mathbf{W}^{(h)}$ s are trainable weights, and $\beta_{ij}^{(h)}$ determines the correlation between $\mathbf{x}_i$ and $\mathbf{x}_j$. To get a general attention value for each user-item feature, we further normalize this value within the feature scope as:

$$\alpha_{ij}^{(h)} = \text{softmax}(\beta_{ij}^{(h)}) = \frac{\exp(\beta_{ij}^{(h)} / \iota)}{\sum_{j=0}^{N-1} \exp(\beta_{ij}^{(h)} / \iota)}, \quad (9)$$

where ι denotes the temperature. We jointly attend on the feature scope from different representation subspaces to learn stably as

$$\omega_i = \sigma \left(\mathbf{W}_q \cdot \left(\frac{1}{H} \sum_{h=0}^{H-1} \sum_{j=0}^{N-1} \alpha_{ij}^{(h)} (\mathbf{x}_j \mathbf{W}_C^{(h)}) \right) + \mathbf{b}_q \right), \quad (10)$$

where H is the number of attention heads, and $\mathbf{W}_\cdot, \mathbf{b}_\cdot$ are trainable parameters. We further integrate this rich information with the attention vector $\mathbf{w}$, to obtain feature embedding $\mathbf{p}$:

$$\mathbf{p} = \frac{1}{N} \sum_{i=0}^{N-1} \mathbf{w}^\top \cdot \tanh(\mathbf{W}_p \cdot \omega_i + \mathbf{b}_p). \quad (11)$$

$\mathbf{p}$ is further fed into two separated MLP modules activated by a sigmoid function without parameter sharing to obtain the estimation of $P(R = 1|O = o, X = \mathbf{x})$ and $P(O = 1|X = \mathbf{x})$ respectively.

Remark. We use Eqs. (8), (9), (10) and (11) to estimate both $P(R = 1|O = o, X = \mathbf{x})$ and $P(O = 1|X = \mathbf{x})$. The only difference is that when predicting the former one, the input is the concatenation of the observation o and user-item features $\mathbf{x}$; while the input is only the user-item features $\mathbf{x}$ when predicting the latter one. However, it does not enforce the requirement of observation information o in raw data, as we can generate $\hat{o}$ to substitute o where $\hat{o} = 1$ when $P(O = 1|X = \mathbf{x}) > 0.5$, and $\hat{o} = 0$ otherwise, as shown in Figure 2. In our experiments, all the observation information is not given.

3.2.2 Estimating C . Given the distribution $P(O = 1|X = \mathbf{x})$ and the conditional distribution $P(R = 1|O = o, X = \mathbf{x})$, we can obtain $P(C|X = \mathbf{x})$ with

$$P(C = 1|X = \mathbf{x}) = P(R = 1|O = o, X = \mathbf{x}) \cdot P(O = 1|X = \mathbf{x}), \quad (12)$$

which can be derived by combining Eqs. (2) and (6).

3.3 Conditional Mutual Information

As discussed in Section 2.2, in typical information retrieval scenarios, *observed* items are likely to be associated with clicks [4, 19, 23], which motivates us to push our relevance estimation conditionally independent to the observation estimation.

Here, we mathematically derive a regularization term of the conditional mutual information formulation according to the following proposition.

PROPOSITION 3.1. *Given that relevance, click, and observation variables are binary (i.e., $R, C, O \in \{1, 0\}$), for any user-item pair with feature X , the following statements are equivalent:*

- *The relevance R and observation O are conditionally independent given X . In other words, $P(R, O|X) = P(R|X) \cdot P(O|X)$. That is, $P(R|O = 1, X) - P(R|O = 0, X) = 0$.*
- *The conditional mutual information between relevance R and observation O (later defined in Eq. (14)) is zero, i.e., $\mathcal{I}(R; O|X) = 0$.*

- The conditional independence score ΔCI is zero.

PROOF. By combining Eqs. (4) and (5), it is trivial to conclude that *statements* 1 and 3 are equivalent.

Here, we further prove that *statements* 1 and 2 are equivalent. According to Eq. (15), *statement* 2 implies $P(R|O = 1, X) = P(R|O = 0, X)$. Given that $O \in \{0, 1\}$, we can derive that $P(R|O, X) = P(R|O = o, X)$ which holds for all $o \in \{0, 1\}$. Hence, we have:

$$\begin{aligned} P(R|X) &= \sum_O P(R|O, X) \cdot P(O|X) \\ &= P(R|O = o, X) \cdot \sum_O P(O|X) = P(R|O, X), \end{aligned} \quad (13)$$

from which we can further derive: $P(R, O|X) = P(R|O, X) \cdot P(O|X) = P(R|X) \cdot P(O|X)$. From the analysis above, we can derive *statement* 1 from *statement* 2. Also, we can obtain *statement* 2 from *statement* 1 in a similar way. $\square$

PROPOSITION 3.1 allows InfoRank to minimize the conditional mutual information I to simply and indirectly enforce the conditional independence between the relevance and observation estimations. I is defined as

$$I := I(R; O|X) = \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} [I(R; O|X = \mathbf{x})], \quad (14)$$

where

$$\begin{aligned} I(R; O|X = \mathbf{x}) &= \sum_{R, O} P(R, O|X = \mathbf{x}) \cdot \ln \frac{P(R, O|X = \mathbf{x})}{P(R|X = \mathbf{x}) \cdot P(O|X = \mathbf{x})} \\ &= \sum_{R, O} P(R|O, X = \mathbf{x}) \cdot P(O|X = \mathbf{x}) \cdot \ln \frac{P(R|O, X = \mathbf{x})}{P(R|X = \mathbf{x})}. \end{aligned} \quad (15)$$

We can further derive $P(R|X = \mathbf{x})$ by $P(R|O, X = \mathbf{x})$ and $P(O|X = \mathbf{x})$ as

$$P(R|X = \mathbf{x}) = \sum_O P(R|O, X = \mathbf{x}) \cdot P(O|X = \mathbf{x}). \quad (16)$$

Note that both $P(R|O, X = \mathbf{x})$ and $P(O|X = \mathbf{x})$ can be obtained from estimations introduced in Section 3.2. Hence, for any given user-item features $\mathbf{x}$, we can calculate the conditional mutual information according to Eqs. (14) (15) and (16).

3.4 Optimization Functions

As shown in Figure 2, our unbiased estimator requires joint optimization of $\mathcal{L}$ and I .

For $\mathcal{L}$, considering that click signals are binary, we adopt a Binary Cross Entropy (BCE) loss for click supervision, which can be formulated as

$$\mathcal{L} = - \sum_{(c, \mathbf{x}) \in \mathcal{D}} \left(c \cdot \log P(\hat{c}|\mathbf{x}) + (1 - c) \cdot \log(1 - P(\hat{c}|\mathbf{x})) \right), \quad (17)$$

where c is the label of the instance, and $\hat{c}$ denotes the corresponding prediction. In some cases where observation information is available, one can easily build a similar loss for observation supervision.

For I , we have already obtained its regularization formulation in Eq. (14). Combining the objective functions together, our goal is

to minimize the log-likelihood of estimation under the conditional mutual information regularization over all the data samples as

$$\arg \min_{\theta} \mathcal{L} + \eta \cdot I, \quad (18)$$

where the hyper-parameter η balances the prediction loss and the regularization and θ denotes all trainable parameters in InfoRank.

3.5 Model Analysis

Unbiased Estimation. As discussed in Section 2.1, unbiased learning-to-rank aims to estimate unbiased relevance r instead of biased click c , whose ideal risk function (denoted as $\tilde{\mathcal{F}}(f)$) can be directly derived from $\mathcal{F}(f)$ (defined in Eq. (1)) by replacing c with r . We show that a ranker is unbiased if it satisfies Eq. (4) in the following proposition.

PROPOSITION 3.2. Assuming that a ranker satisfies Eq. (4), namely $P(R|O = 1, X = \mathbf{x}) - P(R|O = 0, X = \mathbf{x}) = 0$ holds and $P(O|X = \mathbf{x})$ is bounded away from zero for any data point in $\mathcal{D}$, then the ranker is unbiased, namely optimizing

$$\hat{\mathcal{F}}(f) = \sum_u \sum_{d \in \mathcal{D}_u} \frac{\Delta(f(\mathbf{x}), c)}{P(O|X = \mathbf{x})} \quad (19)$$

is equivalent to optimizing $\tilde{\mathcal{F}}(f)$.

PROOF. We can obtain the formulation of the risk function of unbiased ranker from Eq. (1) as

$$\tilde{\mathcal{F}}(f) = \int_u \int_{d \in \mathcal{D}_u} \Delta(f(\mathbf{x}), r) dP(\mathbf{x}, r). \quad (20)$$

However, $\tilde{\mathcal{F}}(f)$ cannot be computed directly, and is typically estimated via the following empirical risk function:

$$\mathcal{F}'(f) = \frac{1}{|\mathcal{D}|} \sum_u \sum_{d \in \mathcal{D}_u} \Delta(f(\mathbf{x}), r), \quad (21)$$

where Δ denotes a point-wise loss function. According to Eq. (12), we can derive that

$$\begin{aligned} \mathbb{E}(\hat{\mathcal{F}}(f)) &= \mathbb{E}\left(\sum \frac{\Delta(f(\mathbf{x}), c)}{P(O|X = \mathbf{x})}\right) = \mathbb{E}\left(\sum \frac{o \cdot \Delta(f(\mathbf{x}), r')}{P(O|X = \mathbf{x})}\right) \\ &= \sum (\mathbb{E}(o) \frac{\Delta(f(\mathbf{x}), r')}{P(O|X = \mathbf{x})}) = \sum (P(O|X = \mathbf{x}) \frac{\Delta(f(\mathbf{x}), r')}{P(O|X = \mathbf{x})}) \\ &= \sum \Delta(f(\mathbf{x}), r') = \mathcal{F}''(f), \end{aligned} \quad (22)$$

where we omit $\frac{1}{|\mathcal{D}|}$ since it is a constant and use $\sum$ to denote $\sum_u \sum_{d \in \mathcal{D}_u}$ for convenience.

Here, r in Eq. (21) is estimated with $P(R|X = \mathbf{x})$, while r' in Eq. (22) is estimated with $P(R|O = o, X = \mathbf{x})$. Since the ranker satisfies Eq. (4), according to Eq. (6) we have:

$$P(R|X = \mathbf{x}) = P(R|O = o, X = \mathbf{x}). \quad (23)$$

So optimizing $\mathcal{F}''(f)$ is equivalent to optimizing $\mathcal{F}'(f)$, implying InfoRank is an unbiased estimator.

Note that we here share some equations with [25]. $\square$

Noticing that Eq. (19) is similar to the risk function of IPW (see Eq. (2) in [4]), our method is more general since we take the effect from the observation factor into consideration. Comparing Eq. (12)

Algorithm 1 InfoRank**INPUT:** implicit feedback dataset $\mathcal{D} = \{(\mathbf{x}, c, o)\}$;**OUTPUT:** unbiased ranker f_θ with parameter θ

```

1: Initialize all parameters.
2: repeat
3:   Randomly sample a batch  $\mathcal{B}$  from  $\mathcal{D}$ 
4:   for each data point  $(\mathbf{x}, c, o)$  in  $\mathcal{B}$  do
5:     Calculate  $P(R = 1|O = o, X = \mathbf{x})$  (and  $P(O = 1|X = \mathbf{x})$ )
       using Eqs. (8), (9), and (11).
6:     Compute  $P(C = 1|X = \mathbf{x})$  using Eq. (12).
7:   end for
8:   Compute  $\mathcal{L}$  and  $\mathcal{I}$  according to Eqs. (17) and (14).
9:   Update  $\theta$  by minimizing Eq. (18).
10: until convergence

```

to Eq. (2), we note that when Eq. (4) is satisfied, our method will reduce to an IPW-based method.

Learning Algorithm. We provide the learning algorithm of InfoRank in Algorithm 1. Note that we calculate the overall objective function as shown in Eq. (18) in the training procedure; while during inference, we compute relevance estimation $P(R|X = \mathbf{x})$ (which corresponds to $P(R|O = o, X = \mathbf{x})$ in the paper) and generate the ranking list in descending order of relevance. It is consistent to the conventional unbiased learning-to-rank pipeline [4, 5, 20]. We also note that although the input data of Algorithm 1 is a set of tuples $\{(\mathbf{x}, c, o)\}$, InfoRank can also be adapted to data without any observation signal, namely $\{(\mathbf{x}, c)\}$, since we can replace the label o from data with the estimation $P(O|X = \mathbf{x})$.

Also, as discussed in [47], our model following a two-tower architecture can not be reduced to a single-tower architecture even with the accessible observation information, as the two-tower architecture would introduce inductive biases to enable more efficient use of limited observational logs and better generalization.

Complexity Analysis. From Section 3.1, the main component of InfoRank is the attention module. Let N denote the total number of user-item features, and H denote the number of attention heads. Then the time complexity of the attention model is $O(NF_1F_2H + EF_1H)$, where E is the number of input feature pairs, and F_1 and F_2 are the number of rows and columns of the attention matrix respectively. Then, the overall complexity is linear regarding the number of features and the number of feature pairs. The prediction network takes $O(C_{\text{att}})$ time where C_{att} is the cost of one multi-head attention operation.

4 EXPERIMENTS

4.1 Datasets and Experimental Setting

Data Description. We evaluate our methods against the strong baselines over three widely adopted learning-to-rank datasets:

- **Yahoo search engine dataset**[¶] is one of the largest benchmark datasets widely used in unbiased learning-to-rank [5, 20, 22].

It consists of 29,921 queries and 710k documents. Each query-document pair is represented by a 700-dimensional feature vector manually assigned with a label denoting relevance at 5 levels [8].

- **LETOR webpage ranking dataset**[‡] is a package of benchmark datasets for research on Learning To Rank, which uses the Gov2 web page collection and two query sets from the Million Query track of TREC2007 and TREC2008. We conduct experiments on MQ2007, one of two datasets for supervised learning-to-rank tasks in LETOR. MQ2007 contains 2,476 queries and 85K documents. Each query-document pair is represented by a 46-dimensional feature vector with a manually assigned 3-level label [29].
- **Adressa recommender system dataset**^{**} is a news dataset that includes news articles in connection with anonymized users, where we can regard user and item information as query and document features respectively. Similar approaches can be found in [22, 41]. It contains 2,287K historical logs of 561,733 users and 11,207 articles. Each query-document pair is represented with several contextual attributes, such as title and category [17]. Following [7], we first calculate the normalized reading time $nv_{u,d}$ with respect to both users' reading habits and the length of different articles:

$$nv_{u,d} = \frac{v_{u,d}}{\mu_d}, \quad (24)$$

where $v_{u,d}$ is the time user u spent reading article d , and μ_d is the average reading time for article d . Then we convert all $nv_{u,d}$ to binary relevance by mapping the top 20% to 1 and the rest to 0.

Let y represent the relevance level in each dataset, then $y \in \{0, 1\}$ for Adressa Recommender system dataset, $y \in \{0, 1, 2\}$ for LETOR webpage ranking dataset, and $y \in \{0, 1, 2, 3, 4\}$ for Yahoo search engine dataset. Following [4, 20], we transform the relevance into a binary space by assigning a threshold on the probability of relevance $P(r = 1)$ which is calculated by

$$P(r = 1) = \epsilon + (1 - \epsilon) \cdot \frac{2^y - 1}{2^{y_{\max}} - 1}, \quad (25)$$

where ϵ denotes click noise and is set to 0.1 by default.

Click Data Generation. In order to simulate the different user's browsing patterns, we follow the click data generation process in [4, 20, 22, 36]. First, we train a Rank SVM model with 1% of the training data with relevance labels. Next, we create an initial ranked list for each query with the trained model. Then, we simulate user browsing processes and sample clicks from the initial list. To make sure InfoRank captures diverse user browsing patterns, we use three simulation models, namely **PBM** [32], **UBM** [14], **CCM** [18], each of which corresponds to a specific browsing pattern:

- **PBM** [32] simulates user browsing behavior based on the assumption that the bias of an item only depends on its position, which can be formulated as $P(o_i) = \rho_i^\tau$, where ρ_i represents position bias at position i and $\tau \in [0, +\infty]$ is a parameter controlling the degree of position bias. The position bias ρ_i is obtained from an eye-tracking experiment in [24] and the parameter τ is set as 1 by default. It also assumes that a user decides to click a document d_i according to the probability $P(c_i) = P(o_i) \cdot P(r_i)$.

[‡]<https://www.microsoft.com/en-us/research/project/letor-learning-rank-information-retrieval/>

^{**}<http://reclab.idi.ntnu.no/dataset/>

[¶]<http://webscope.sandbox.yahoo.com>

Table 2: Comparison of different unbiased learning-to-rank methods on Yahoo Search Engine, LETOR Webpage Ranking, and Adressa Recommender System datasets. UBM is used as a click generation model. * indicates $p < 0.001$ in significance tests compared to the best baseline.

Ranker	Debiasing Method	Yahoo (UBM)				LETOR (UBM)				Adressa (UBM)			
		MAP	N@3	N@5	N@10	MAP	N@3	N@5	N@10	MAP	N@3	N@5	N@10
InfoRank (Ranking)	Labeled Data	.856	.755	.760	.795	.695	.381	.468	.563	.821	.714	.727	.754
	InfoRank (Debiasing)	.845*	.736*	.739*	.779*	.650*	.380*	.460*	.541*	.801*	.691*	.715*	.739*
	Regression-EM	.837	.683	.692	.731	.634	.374	.442	.535	.794	.673	.706	.731
	Randomization	.835	.680	.689	.728	.630	.368	.437	.515	.792	.668	.695	.728
	Click Data	.823	.670	.678	.720	.622	.356	.428	.489	.782	.648	.677	.707
LambdaMART	Labeled Data	.854	.745	.757	.790	.685	.380	.461	.558	.814	.709	.722	.747
	Ratio-Debiasing	.832	.712	.722	.755	.631	.365	.421	.506	.791	.669	.702	.730
	Regression-EM	.827	.680	.693	.741	.628	.356	.411	.490	.785	.650	.681	.711
	Randomization	.824	.675	.687	.725	.624	.346	.407	.482	.784	.648	.678	.705
	Click Data	.814	.666	.673	.712	.614	.339	.396	.473	.779	.635	.664	.694
DNN	Labeled Data	.831	.685	.705	.737	.678	.364	.454	.551	.802	.700	.722	.745
	InfoRank (Debiasing)	.828	.683	.696	.734	.637	.360	.416	.499	.786	.667	.692	.725
	Dual Learning	.825	.680	.693	.730	.625	.352	.410	.487	.784	.663	.688	.720
	Regression-EM	.823	.676	.689	.726	.618	.347	.400	.479	.779	.656	.675	.713
	Randomization	.822	.677	.686	.724	.617	.346	.397	.477	.777	.644	.664	.701
	Click Data	.817	.665	.671	.710	.612	.335	.387	.469	.775	.633	.659	.688

- **UBM** [14] is an extension of the PBM model that has some elements of the cascade model. The examination probability depends not only on the rank of an item d_i but also on the rank of the previously clicked document $d_{i'}$ as $P(o_i = 1|c_{i'} = 1, c_{i'+1} = 0, \dots, c_{i-1} = 0) = \gamma_0$. Similarly, we get γ_0 from the eye-tracking experiments in [14, 24]. The click probability is determined by $P(c_i) = P(o_i) \cdot P(r_i)$.
- **CCM** [18] is a cascade model, which assumes that the user browses search results in a sequential order from top to bottom. User browsing behavior is conditioned on both current and past documents, as $P(c_i = 1|o_i = 0) = 0$, $P(c_i = 1|o_i = 1, r_i) = P(r_i)$, $P(o_{i+1} = 1|o_i = 0) = 0$, $P(o_{i+1} = 1|o_i = 1, c_i = 0) = \gamma_1$, $P(o_{i+1} = 1|o_i = 1, c_i = 1, r_i) = \gamma_2 \cdot (1 - P(r_i)) + \gamma_3 \cdot P(r_i)$. The parameters are obtained from an experiment in [18] with $\gamma_2 = 0.10$ and $\gamma_3 = 0.04$ for navigational queries (Yahoo and LETOR datasets); $\gamma_2 = 0.40$ and $\gamma_3 = 0.27$ for informational queries (Adressa dataset). γ_1 is set to 0.5 by default.

Hyperparameter Setting. Our experiments were implemented with Tensorflow. From the baseline methods, we directly use their default hyper-parameters. For InfoRank, we set the learning rate as 0.001, batch size as 128, batch normalization decay as 0.9, the weight of α as 0.01, L2 regularization weight as 0.01, and optimize with Adam. In the main experiment, we set η as 0.5. All the models are trained under the same hardware settings with 16-Core AMD Ryzen 9 5950X (2.194GHZ), 62.78GB RAM, NVIDIA GeForce RTX 3080 cards.

All code will be released at publication time.

4.2 Baselines and Metrics

Baselines are created by combining the learning-to-rank algorithm with state-of-the-art debiasing methods. We perform comparisons against strong baselines with the debiasing methods, introduced as follows. **Randomization** [25] is a randomization technique to infer the observation probabilities. While practical, we randomly

shuffled the rank lists and then estimated the position biases as in [4, 20]. **Regression-EM** [44] is a regression-based expectation maximization (EM) method, where position bias is directly estimated from regular production clicks. **Dual Learning** [4] is a dual learning algorithm, which can jointly learn a debiasing model and a ranker on click data. **Ratio-Debiasing** [20] is a pair-wise unbiased learning-to-rank model based on inverse propensity weight (IPW) [43]. Let **InfoRank (Debiasing)** denote the proposed debiasing method based on the conditional mutual information minimization. We use **Click Data** to represent methods directly built on raw click data without debiasing to train the ranker, whose performance is regarded as a lower bound. **Labeled Data** represents methods where a ranker is trained directly on human-annotated relevance labels without any bias, whose performance is an upper bound. There are several learning-to-rank algorithms that can be combined with debiasing methods: **DNN** is a deep neural network ranker as described in [4]. **LambdaMART** is a widely used learning-to-rank algorithm as described in [6]. We further introduce InfoRank⁻ to denote the variant of InfoRank without the conditional mutual information regularization.

We use **InfoRank (Ranking)** to denote the proposed attention-based learning-to-rank method. Following the experimental settings in [20], we do not combine Ratio-Debiasing with DNN and Dual Learning with LambdaMART, since DNN is originally adopted in Dual Learning and Ratio-Debiasing are typically designed based on LambdaMART. We also incorporate InfoRank (Debiasing) into DNN by using DNN as our estimations instead of the proposed attention network.

We evaluate the above methods on unbiased learning-to-rank tasks. In order to evaluate the performance of each method, we choose NDCG at position 3, 5, 10 (denoted as N@3, N@5, N@10) and MAP at position 10 (denoted as MAP) as evaluation measures. Besides the accuracy of the ranking performance, in this paper, we have introduced another evaluation metrics (denoted as ΔCI in Eq. (5)) regarding the casual dependence in Section 2.2.

Table 3: Comparison of different unbiased learning-to-rank methods under PBM, CCM. Yahoo Search Engine data and LETOR Webpage Ranking are used as click datasets. Results of Adressa Recommendation System follow similar trends, and are omitted due for brevity. * indicates $p < 0.001$ in significance tests compared to the best baseline.

Ranker	Debiasing Method	Yahoo (PBM)			Yahoo (CCM)			LETOR (PBM)			LETOR (CCM)		
		MAP	N@5	N@10	MAP	N@5	N@10	MAP	N@5	N@10	MAP	N@5	N@10
InfoRank (Ranking)	Labeled Data	.856	.760	.795	.856	.760	.795	.695	.468	.563	.695	.468	.563
	InfoRank (Debiasing)	.849*	.732*	.772*	.846*	.712*	.758*	.681*	.457*	.559*	.658*	.455*	.539*
	Regression-EM	.841	.715	.740	.822	.685	.734	.675	.453	.552	.652	.450	.534
	Randomization	.840	.704	.736	.817	.679	.728	.671	.450	.551	.649	.449	.531
	Click Data	.831	.682	.725	.808	.658	.710	.647	.445	.510	.640	.439	.498
LambdaMART	Labeled Data	.854	.757	.790	.854	.757	.790	.685	.461	.558	.685	.461	.558
	Ratio-Debiasing	.836	.728	.764	.828	.691	.738	.648	.446	.513	.644	.440	.502
	Regression-EM	.830	.700	.743	.816	.675	.727	.636	.436	.509	.634	.431	.497
	Randomization	.827	.690	.728	.814	.673	.722	.633	.433	.498	.628	.427	.493
	Click Data	.820	.672	.716	.804	.653	.706	.630	.424	.494	.625	.418	.488
DNN	Labeled Data	.831	.705	.737	.831	.705	.737	.678	.454	.551	.678	.454	.551
	InfoRank (Debiasing)	.829	.703	.736	.828	.692	.735	.651	.446	.547	.650	.444	.531
	Dual Learning	.828	.697	.734	.823	.681	.731	.645	.437	.528	.638	.430	.525
	Regression-EM	.829	.699	.736	.819	.678	.728	.635	.426	.500	.628	.417	.490
	Randomization	.825	.693	.732	.816	.674	.726	.630	.419	.495	.625	.415	.487
	Click Data	.819	.667	.711	.801	.650	.705	.629	.419	.492	.621	.409	.480

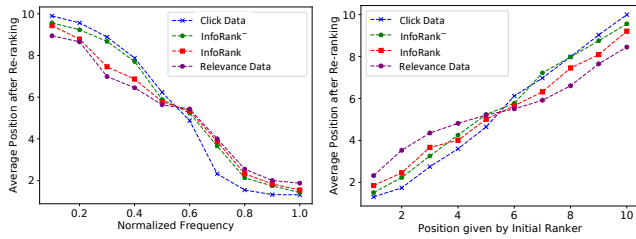


Figure 3: Average positions after re-ranking of items at each normalized frequency (in the left subfigure); or at each original position (in the right subfigure) by different debiasing methods together with InfoRank and InfoRank⁺ on Yahoo.

4.3 Performance Comparison

We summarize the results regarding the ranking performance in Table 2. The results of Ratio-Debiasing, Regression-EM and Dual Learning are comparable with those reported in [20]. We show the major findings as follows:

- Our model that combines InfoRank (Ranking) and InfoRank (Debiasing) together achieves better performance than any combinations of the existing state-of-the-art ranker and debiasing methods (e.g., the combination of LambdaMART with Ratio-Debiasing, the combination of DNN with Dual Learning) in terms of all measures, which verifies the superiority of our framework.
- We find that although sophisticated ranking algorithms like InfoRank (Ranking), LambdaMART and DNN can achieve good performance on click data, they are sensitive to position bias when comparing against the performance of debiasing methods, which indicates the importance of unbiased learning-to-rank.
- When using the same ranker, InfoRank (Debiasing) consistently outperforms other debiasing methods; and when under the same debiasing method (e.g., Regression-EM, Randomization), InfoRank (Ranking) could achieve the best performance.

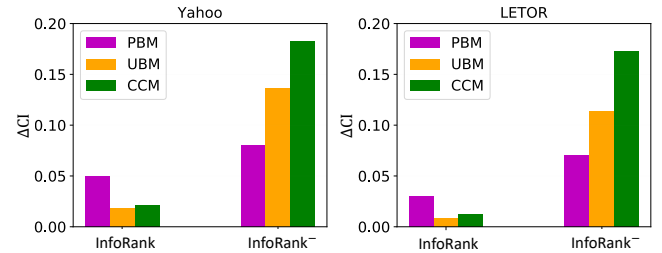


Figure 4: Comparison of InfoRank and InfoRank⁺ under different click generation models and datasets in terms of the ΔCI metric.

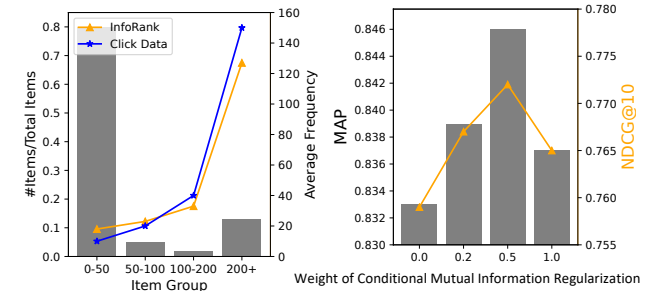


Figure 5: Left: Average frequency of different item groups recommended by InfoRank (Ranking) incorporated with InfoRank (Debiasing) and Click Data on Adressa. Right: Performance change of InfoRank with different regularization weight η on Yahoo.

4.4 Debiasing Analysis

Visualization Analysis of Popularity Bias. For popularity bias, since there is no unique ID for each item, we first adopt the K-means algorithm where item similarity is measured by Pearson’s coefficient. We use the query number as the number of classes, and regard items in the same class as the same item. We then study the correlations between the item frequency and the position of

re-ranking lists. As depicted in Figure 3(Left), the curve of click data (in blue with mark ‘x’) is far from that of relevance labels (in purple with mark ‘•’), indicating that directly using click data without debiasing can be problematic. The curve of InfoRank (in red with ‘■’) is the closest to the relevance label curve, indicating that the performance enhancement of InfoRank is indeed, at least partly, due to effective debiasing.

Visualization Analysis of Position Bias. For position bias, we compare the ranking list given by the debiased ranker against the initial one. Specifically, we first identify the items at each position given by the initial ranker. Then we calculate the average positions of items at each original position after re-ranking by various debiasing methods, combined with InfoRank. We also calculate the their average positions after re-ranking by their relevance labels, which is regarded as the ground-truth. Ideally the average positions produced by the debiasing methods should be close to the average position by relevance labels. Similar to the visualization analysis of popularity bias, Figure 3(Right) shows that the curve of InfoRank (in red with ‘■’) is the closest to the relevance label curve, revealing that InfoRank could truly decline the effect of position bias.

Ablation Study on ΔCI Metric. We further conduct evaluations under different simulators and datasets in terms of the ΔCI metric. From results shown in Figure 4, we can see that InfoRank with the conditional mutual information regularization works better than InfoRank without it in terms of ΔCI metric. We also notice that the regularization term shows weaker impact on PBM than UBM and CCM. One explanation is that PBM directly assumes the observation is solely determined by the position instead of user-item feature, namely the observation is naturally independent of the relevance. By contrast, given the formulation of UBM and CCM, the observation is dependent on the click, and thus further is influenced by user-item feature.

Ablation Study of Item Groups with Different Frequency. We follow [45] to further investigate whether InfoRank alleviates the popularity bias issue. Table 2 shows InfoRank (Debiasing) significantly outperforms Click Data, but Figure 5(Left) shows the recommendation frequency of popular items is reduced. It means that the models trained by Click Data are prone to recommend more popular items to unrelated users due to popularity bias. In contrast, InfoRank (Debiasing) reduces the item’s direct effect and recommends popular items mainly to suitable users.

4.5 Robustness Analysis

Hyperparameter Study of η . To evaluate how the weight of the regularization η influences the performance of InfoRank, we set $\eta = 0.0, 0.2, 0.5, 1.0$ and test the performance on Yahoo. Results depicted in Figure 5(Right) imply that the proposed mutual information minimization can enhance the ranking performance.

Generalizability to Different Click Generations. To test the generalizability of InfoRank, we use three different click generation models to generate data and conduct comparison experiments against the baselines on these data. Each click model simulates a certain user browsing pattern. InfoRank has strong performance

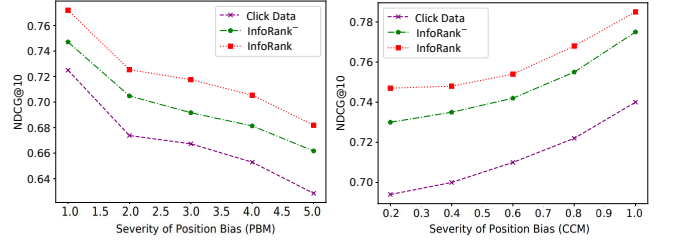


Figure 6: Performance of InfoRank against click data with different degrees of position bias on Yahoo.

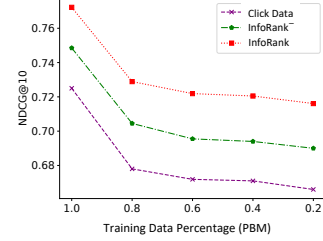


Figure 7: Performance of InfoRank and InfoRank- against click data with different amounts of training data on Yahoo.

on data generated with UBM (as shown in Table 2), and also outperforms baselines on the data generated from PBM and CCM (as shown in Table 3), which shows that the idea of InfoRank is general to various user patterns.

Different Degrees of Position Bias. In the above experiments, we only test the performance of InfoRank and InfoRank- with click data generated from click models with a given degree of position bias, i.e., $\gamma_1 = 0.5$ in CCM and $\tau = 1$ in PBM. Here, γ_1 and τ influence the probability of a user examining the next result. Smaller γ_1 and larger τ indicate a smaller probability to continue reading, which means a more severe position bias. Therefore, we set the hyper-parameters for each click generation model to five values and examine whether InfoRank is still equally effective. The left and right subfigures of Figure 6 show the NDCG@10 results as the degree of position bias increases; the results in terms of other measures follow similar trends. When τ in PBM is 0, there is no position bias; while γ_1 in CCM is 1, there still exists position bias from γ_2 and γ_3 . As we add more position bias, i.e., as τ increases and γ_1 decreases, the performance of all the debiasing methods decreases dramatically. One can see that under all these settings, InfoRank is less affected by position bias and consistently maintains the best results.

Different Amounts of Training Data. We study the robustness of InfoRank with or without the conditional mutual information regularization, denoted as InfoRank or InfoRank-, under different amounts of training data on Yahoo. We first randomly select a subset of training data (i.e., 20% - 100%) to generate click data, and then use these datasets to train InfoRank with different debiasing methods. For fair comparison, we use the same data for evaluation across all experiments. As shown in Figure 7, when the amount of training data decreases, the improvements obtained by the debiasing methods also decrease. The reason could be that the position bias estimated from insufficient training data is not accurate, which can hurt the performance of debiasing methods.

5 CONCLUSION AND FUTURE WORK

In this paper, we propose a novel unbiased learning-to-rank framework named InfoRank. The key insight of our work is to unify the effects of different biases into a single observation factor. It enables us to simultaneously learn an accurate ranker and address the position bias and popularity bias. For future work, it would be interesting to generalize InfoRank to other ranking biases.

REFERENCES

- [1] Himan Abdollahpour, Robin Burke, and Bamshad Mobasher. 2017. Controlling popularity bias in learning-to-rank recommendation. In *RecSys*.
- [2] Aman Agarwal, Kenta Takatsu, Ivan Zaitsev, and Thorsten Joachims. 2019. A general framework for counterfactual learning-to-rank. In *SIGIR*.
- [3] Aman Agarwal, Ivan Zaitsev, Xuanhui Wang, Cheng Li, Marc Najork, and Thorsten Joachims. 2019. Estimating position bias without intrusive interventions. In *WSDM*.
- [4] Qingyao Ai, Keping Bi, Cheng Luo, Jiafeng Guo, and W Bruce Croft. 2018. Unbiased learning to rank with unbiased propensity estimation. In *SIGIR*.
- [5] Qingyao Ai, Jiaxin Mao, Yiqun Liu, and W Bruce Croft. 2018. Unbiased learning to rank: Theory and practice. In *CIKM*.
- [6] Christopher JC Burges. 2010. From ranknet to lambdarank to lambdamart: An overview. *Learning* (2010).
- [7] Abhijnan Chakraborty, Gourab K Patro, Niloy Ganguly, Krishna P Gummedi, and Patrick Loiseau. 2019. Equality of voice: Towards fair representation in crowd-sourced top-k recommendations. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*.
- [8] Olivier Chapelle and Yi Chang. 2011. Yahoo! learning to rank challenge overview. In *Proceedings of the learning to rank challenge*.
- [9] Olivier Chapelle and Ya Zhang. 2009. A dynamic bayesian network click model for web search ranking. In *WWW*.
- [10] Jiawei Chen, Hande Dong, Feng Wang, Xiang, Wang Fuli, He Meng, and Xiangnan. 2020. Bias and Debias in Recommender System: A Survey and Future Directions. *arXiv preprint (2020)*.
- [11] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishu Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Isipir, et al. 2016. Wide & deep learning for recommender systems. In *DLRS*.
- [12] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *RecSys*.
- [13] Nick Craswell, Onno Zoeter, Michael Taylor, and Bill Ramsey. 2008. An experimental comparison of click position-bias models. In *WSDM*.
- [14] Georges E Dupret and Benjamin Piwowarski. 2008. A user browsing model to predict search engine click data from past observations. In *SIGIR*.
- [15] Michael D Ekstrand, Mucun Tian, Ion Madrazo Azpiazu, Jennifer D Ekstrand, Oghenemaro Anuyah, David McNeill, and Maria Soledad Pera. 2018. All the cool kids, how do they fit in?: Popularity and demographic biases in recommender evaluation and effectiveness. In *Conference on Fairness, Accountability and Transparency*.
- [16] Zhichong Fang, Aman Agarwal, and Thorsten Joachims. 2019. Intervention harvesting for context-dependent examination-bias estimation. In *SIGIR*.
- [17] Jon Atle Gulla, Lemei Zhang, Peng Liu, Özlem Özgöbek, and Xiaomeng Su. 2017. The Adressa dataset for news recommendation. In *Proceedings of the international conference on web intelligence*.
- [18] Fan Guo, Chao Liu, Anitha Kannan, Tom Minka, Michael Taylor, Yi-Min Wang, and Christos Faloutsos. 2009. Click chain model in web search. In *WWW*.
- [19] Katja Hofmann, Anne Schuth, Alejandro Bellogin, and Maarten De Rijke. 2014. Effects of position bias on click-based recommender evaluation. In *ECIR*.
- [20] Ziniu Hu, Yang Wang, Qu Peng, and Hang Li. 2019. Unbiased LambdaMART: An unbiased pairwise learning-to-rank algorithm. In *WWW*.
- [21] Rolf Jagerman, Harrie Oosterhuis, and Maarten de Rijke. 2019. To model or to intervene: A comparison of counterfactual and online learning to rank from user interactions. In *SIGIR*.
- [22] Jiarui Jin, Yuchen Fang, Weinan Zhang, Kan Ren, Guorui Zhou, Jian Xu, Yong Yu, Jun Wang, Xiaoqiang Zhu, and Kun Gai. 2020. A Deep Recurrent Survival Model for Unbiased Ranking. In *SIGIR*.
- [23] Thorsten Joachims, Laura Granka, Bing Pan, Helene Hembrooke, and Geri Gay. 2017. Accurately interpreting clickthrough data as implicit feedback. In *SIGIR*.
- [24] Thorsten Joachims, Laura A Granka, Bing Pan, Helene Hembrooke, and Geri Gay. 2005. Accurately interpreting clickthrough data as implicit feedback. In *SIGIR*.
- [25] Thorsten Joachims, Adith Swaminathan, and Tobias Schnabel. 2017. Unbiased learning-to-rank with biased feedback. In *WSDM*.
- [26] Toshihiro Kamishima, Shotaro Akaho, Hideki Asoh, and Jun Sakuma. 2014. Correcting Popularity Bias by Enhancing Recommendation Neutrality. In *RecSys Posters*.
- [27] Adit Krishnan, Ashish Sharma, Aravind Sankar, and Hari Sundaram. 2018. An adversarial approach to improve long-tail performance in neural collaborative filtering. In *CIKM*.
- [28] Harrie Oosterhuis and Maarten de Rijke. 2021. Unifying Online and Counterfactual Learning to Rank: A Novel Counterfactual Estimator that Effectively Utilizes Online Interventions. In *WSDM*.
- [29] Tao Qin, Tie-Yan Liu, Jun Xu, and Hang Li. 2010. LETOR: A benchmark collection for research on learning to rank for information retrieval. *Information Retrieval* (2010).
- [30] Yanru Qu, Han Cai, Kan Ren, Weinan Zhang, Yong Yu, Ying Wen, and Jun Wang. 2016. Product-based neural networks for user response prediction. In *ICDM*.
- [31] Steffen Rendle. 2010. Factorization machines. In *ICDM*.
- [32] Matthew Richardson, Ewa Dominowska, and Robert Ragno. 2007. Predicting clicks: estimating the click-through rate for new ads. In *WWW*.
- [33] Paul R Rosenbaum and Donald B Rubin. 1983. The central role of the propensity score in observational studies for causal effects. *Biometrika* (1983).
- [34] Ying Shan, T Ryan Hoens, Jian Jiao, Haijing Wang, Dong Yu, and JC Mao. 2016. Deep crossing: Web-scale modeling without manually crafted combinatorial features. In *KDD*.
- [35] Harald Steck. 2019. Collaborative filtering via high-dimensional regression. *arXiv preprint arXiv:1904.13033* (2019).
- [36] Anh Tran, Tao Yang, and Qingyao Ai. 2021. ULTRA: An Unbiased Learning To Rank Algorithm Toolbox. In *CIKM*.
- [37] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *NeurIPS*.
- [38] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2018. Graph attention networks. *ICLR* (2018).
- [39] Chao Wang, Yiqun Liu, Meng Wang, Ke Zhou, Jian-yun Nie, and Shaoping Ma. 2015. Incorporating non-sequential behavior into click models. In *SIGIR*.
- [40] Hongning Wang, ChengXiang Zhai, Anlei Dong, and Yi Chang. 2013. Content-aware click modeling. In *WWW*.
- [41] Jun Wang, Arjen P De Vries, and Marcel JT Reinders. 2006. A user-item relevance model for log-based collaborative filtering. In *ECIR*.
- [42] Nan Wang, Xuanhui Wang, and Hongning Wang. 2020. Unbiased Learning to Rank via Propensity Ratio Scoring. In *SIGIR*.
- [43] Xuanhui Wang, Michael Bendersky, Donald Metzler, and Marc Najork. 2016. Learning to rank with selection bias in personal search. In *SIGIR*.
- [44] Xuanhui Wang, Nadav Golbandi, Michael Bendersky, Donald Metzler, and Marc Najork. 2018. Position bias estimation for unbiased learning to rank in personal search. In *WSDM*.
- [45] Tianxin Wei, Fuli Feng, Jiawei Chen, Ziwei Wu, Jinfeng Yi, and Xiangnan He. 2021. Model-agnostic counterfactual reasoning for eliminating popularity bias in recommender system. In *KDD*.
- [46] Xinwei Wu, Hechang Chen, Jiashu Zhao, Li He, Dawei Yin, and Yi Chang. 2021. Unbiased Learning to Rank in Feeds Recommendation. (2021).
- [47] Le Yan, Zhen Qin, Honglei Zhuang, Xuanhui Wang, Mike Bendersky, and Marc Najork. 2022. Revisiting two tower models for unbiased learning to rank. (2022).
- [48] Xing Yi, Liangjie Hong, Erheng Zhong, Nanthan Nan Liu, and Suju Rajan. 2014. Beyond clicks: dwell time for personalization. In *RecSys*.
- [49] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *KDD*.
- [50] Ziwei Zhu, Yun He, Xing Zhao, and James Caverlee. 2021. Popularity Bias in Dynamic Recommendation. (2021).